

The Three Principles of LSJ: Infinite Expansion, Dynamic Evolution, and Real-World Integration

DKI-7
xiaohua@dkl-7.com

Abstract

“World creation” has long been a human dream. Almost every child has imagined building a world of their own, yet such a vision has never truly been realized. Recent advances in AI-generated content have renewed this possibility. Existing generative systems can already produce text, images, audio, video, 3D assets, and other forms of content, but these capabilities still largely remain isolated from one another: they create “content about a world” rather than the world itself. Recent world models represent a significant step forward, but most current systems still mainly excel at continuous visual generation and limited interaction prediction. Their real-time responsiveness and capacity for open-ended interaction remain insufficient, leaving a clear gap before they can support a world that responds immediately, runs for the long term, and can be continuously changed by users. A believable persistent world also requires spatial organization, rule constraints, long-term memory, multi-user collaboration, character relationships, physical feedback, runtime mechanisms, and continuous narrative progression. DKI-7, the Dream Guardian Research Institute No. 7, proposes an architectural framework for worlds that can expand without limit, evolve dynamically, and support real-world integration.

This paper presents LSJ (“Inner World”), a persistent-world architecture built around five core capability pillars: layered world intelligence, dynamic multimodal and multi-type content generation with runtime integration, a distributed network of autonomous digital federation nodes, a continuously streamed open world and reliable runtime framework, and the continuous evolution of world structure, runtime mechanisms, and narrative progression. Unlike conventional content-generation systems, LSJ focuses not merely on generating assets, but on whether generated results can truly become part of the world: whether they can persist over time, be jointly changed by multiple users, support physically grounded interaction, be recorded in world history, and continue to shape characters, regions, events, and social relationships in the future.

In addition, this paper frames real-world integration as a system-level feedback loop. Real-world needs, organizations, identity permissions, activity processes, and external events are incorporated into LSJ’s spatial, object, task, rule, and chronicle memory systems after semantic understanding and transformation through a unified intermediate representation. Collaboration, transactions, task progression, and state changes inside LSJ can also feed back into and support real-world organizations, activities, and collaborative workflows in return.

The core view of this paper is that open-world intelligence should not be viewed simply as replacing a traditional engine with a single large-scale model. Instead, it emerges from coordination among model capabilities, world memory, generation systems, network hosting, and engineering runtime infrastructure. DKI-7’s goal is not merely to generate impressive content, but to make content truly become part of a living world, allowing users to perceive that the world can be created, entered, changed, preserved, and continuously extended.

Keywords: open-world intelligence; world models; dynamic content generation; chronicle memory system; semantic addressing system; distributed network hosting; real-world integration; reliable runtime framework

1 Introduction

1.1 Research Background

From mythology, literature, film, games, and virtual reality, humans have always searched for new media for constructing worlds. Traditional media can describe worlds, games can let people enter bounded or localized virtual worlds, and generative AI has further lowered the threshold for content production. However, generating an image, an asset, or a story fragment is not equivalent to generating a world that can run for the long term.

A world has a more complex structure than content. It includes space, rules, characters, history, social relationships, state changes, and shared memory. Users do not merely watch a world; they also want to enter it, change it, build it together with others, and see their actions continue to have effects in the future. Against this background, DKI-7 proposes LSJ as a system architecture for continuous world generation and runtime operation.

1.2 Limitations of Current Content-Generation Systems

Current content-generation systems and explorations around world models are already very strong in local capabilities. Text models are good at narration, image models are good at visual expression, and 3D generation tools can produce initial asset drafts [13, 17, 27]. Agent systems can complete local conversations or tasks. Some world models have also begun to learn abstract state representations from images, videos, and physical dynamics, or to generate controllable interactive environments from visual input [4–7, 12, 21]. Overall, however, these capabilities remain difficult to integrate into a persistent world system.

This limitation appears mainly in five aspects. First, many generated results still remain at the static-content layer and lack long-term runtime state. Second, although current world-model research is rapidly advancing in representation learning, video prediction, interactive environment generation, and planning, it still falls short of the real-time, scalable, and multi-type interaction required by open worlds [5–7, 12, 16, 21]. Even when some systems already provide nearly real-time visual output, their interaction mechanisms are usually still limited by predefined action spaces and training distributions, making it difficult to rapidly add highly diverse interaction rules, object abilities, and interaction mechanisms. Third, there is no unified world semantics across modalities, making text, images, assets, and runtime mechanisms difficult to align naturally. Fourth, multi-user collaboration often means co-editing content rather than jointly changing a persistent world. Fifth, NPCs and world events lack sufficiently stable historical accumulation and easily become one-time conversations or localized narrative episodes.

These are precisely the systemic problems that DKI-7 hopes to solve through LSJ. LSJ does not treat generation capabilities as isolated tools. Instead, it organizes generation, runtime, memory, collaboration, and evolution into a single world framework.

1.3 The Vision of LSJ and the Three Principles of World Creation

DKI-7 defines LSJ as a system architecture in which the world is the basic object of generation and operation, and describes its long-range outlook through three principles. First, the world supports infinite

expansion: in theory, the world can keep growing, and its spatial boundaries are jointly determined by resource conditions, hosting methods, and runtime strategies. Second, the world is persistently evolving: life forms, characters, organizations, and events in the world have their own trajectories, and participants can enter the world state through different identities or perspectives. Third, the world supports real-world integration: activities, organizations, needs, and identities in real life can enter the world and maintain long-term interactions with the virtual world.

Here, real-world integration does not mean simply transferring real-world materials into virtual space. Rather, it means interpreting real-world information, structuring it, and grounding it into executable world representations. Digital-twin research has emphasized the relationship among physical entities, virtual models, and persistent data mapping, and recent work has begun to explore large language models for continuously updating digital twins [2, 30]. In LSJ, real-world organizational relationships, activity schedules, business needs, scene materials, identity permissions, and event feedback need to be translated into spaces, objects, tasks, rules, NPC relationships, and historical records inside the world. When real-world inputs can change world state and world-state changes can in turn support real-world activities or organizational processes, the virtual world becomes an executable extension of real-world activity rather than a display layer.

These three principles are not promotional slogans. They are evaluation criteria for the architectural design that follows.

2 System Overview

LSJ is a system that takes persistent worlds as the primary unit of generation and runtime operation. Its output is a persistent virtual world composed of space, assets, runtime mechanisms, physical feedback, networked hosting and coordination, social structure, NPC memory, and world history.

Overall, LSJ contains five core capability pillars. First, layered world intelligence supports world understanding, long-term memory, generation control, and runtime constraints. Second, dynamic multimodal and multi-type content generation with runtime integration transform user ideas, real-world needs, third-party generation results, and events produced by the world runtime into content that can become part of the world. Third, distributed network hosting for autonomous digital federation nodes allows different users, creators, organizations, and communities to host their own worlds and interconnect under unified public capabilities. Fourth, a continuously streamed open world and reliable runtime framework supports large-scale space, validated physical feedback, and long-term state. Fifth, world structure, runtime mechanisms, and continuous narrative progression allow characters, relationships, organizations, and events to keep changing in the persistent world.

LSJ forms a real-world integration feedback loop. Real-world inputs first enter the system in the form of text, images, organizational materials, business configurations, activity schedules, or external-system events. They are then transformed through semantic addresses, intent parsing, and a unified intermediate representation (IR) into objects, spaces, rules, tasks, and permissions that the world can understand. Generation, validation, network hosting, and runtime systems then integrate these structures into the persistent world. Finally, the chronicle memory system and external ecosystem interfaces record world changes and, when needed, feed them back into real-world organizations, business processes, or collaborative workflows. This feedback loop draws on the digital-twin idea of connecting physical entities, virtual models, and persistent data flows, platform practices such as NVIDIA Omniverse, and recent attempts to use large language models for context-adaptive digital-twin updates [2, 20, 23, 30]. Rather than copying real objects, it establishes a continuous mapping between real-world state and world state.

Figure 2-1 contains several core components. The semantic addressing system converts user input into

compact semantic routing representations that support routing, compression, and scheduling; this design is related to recent work on semantic routing for LLM-assisted systems [1, 24]. The unified intermediate representation (IR) converts text, images, real-world needs, and world events into object, rule, space, and task structures that engineering systems can consume. LSJ’s world intelligence is a layered intelligence system composed of latent world-state inference, large-language-model capabilities, the chronicle memory system, and lightweight model collaboration. Low-frequency decision layers handle world understanding, structural inference, and long-term semantic judgment, while high-frequency runtime decision layers handle local, repetitive, low-latency runtime judgment. Engineering systems are responsible for authoritative state, validation, and execution, while the chronicle memory system records world history. LSJS, UGCS, and LSJC correspond respectively to official public support nodes, user-autonomous hosting nodes, and user-terminal access nodes, forming the distributed world network described later.

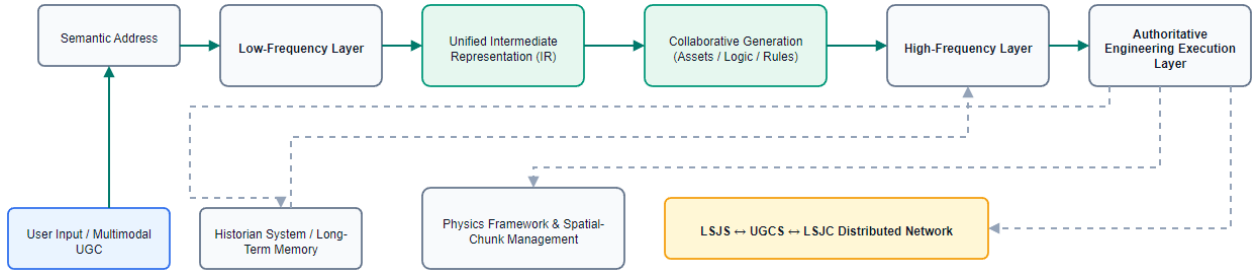


Figure 2-1: Overall LSJ Architecture

Figure 2-1 summarizes LSJ’s end-to-end architecture and data-flow relationships from user input to world operation. Multimodal user input is encoded and routed by the semantic addressing system before being converted into the unified intermediate representation (IR). The layered world-intelligence system provides high-level understanding at low frequency. Small Model Farm (SMF), a lightweight model collaboration system, provides local acceleration at high frequency. The engineering system provides authoritative execution, while the chronicle memory system continuously records world history. In the distributed network, official public support nodes (LSJS), user-autonomous hosting nodes (UGCS), and user-terminal access nodes (LSJC) jointly support the long-term hosting and evolution of the open world.

3 Layered World Intelligence: Latent World-State Model, Chronicle Memory System, and Model Capability System

3.1 Chapter Overview

Building on Chapter 2’s summary of the layered world-intelligence system, this chapter further explains the division of labor among large-scale models, long-term memory systems, lightweight auxiliary models, and engineering runtime systems. LSJ does not use large-scale models as the sole executor of the world. Instead, models are responsible for understanding, state completion, interpretation, and future-state pre-evaluation, while engineering runtime systems are responsible for state consistency, rule validation, and real-time execution.

The chapter also describes how the visual-expression layer, standardized engineering models, and semantic addressing system participate in layered world intelligence. The former ensures differentiated perception under the same world logic, while the latter two provide stable support for model invocation, candidate evaluation, and runtime integration.

3.2 Layered World Intelligence

LSJ’s model capabilities can be summarized into three levels. The first is the long-term semantic layer, which understands world background, character relationships, regional history, and user intent. The second is the regional state layer, which maintains change trends, event impacts, and short-term situations in local worlds. The third is the lightweight auxiliary layer, which provides stable and fast localized decision-making in high-frequency interactions.

LSJ’s model capabilities do not replace the runtime system. Instead, they form a division of labor with the engineering execution layer. Models are mainly responsible for understanding, generation, state completion, and approximate local decision-making; the engineering layer is responsible for authoritative execution.

This is especially important in high-frequency real-time interaction. Interaction input, physics, animation, UI, and other links require low latency and determinism. The unpredictable inputs brought by large amounts of UGC or generated content in an open world further highlight the importance of engineering-layer constraints, state maintenance, and exception handling.

In the LSJ system, the engineering layer carries core responsibilities, including:

- **Authoritative state maintenance:** Ensures consistency between world state and object state, keeping operations correct from a global perspective.
- **Real-time execution:** Meets frame-rate and latency requirements for high-frequency interactions and supports immediate open-world response.
- **Constraint validation:** Checks behavioral validity, collision results, navigation paths, and resource constraints to ensure runtime operations comply with system rules.
- **Fallback and exception recovery:** Rolls back model outputs that may be unstable or out of bounds to prevent abnormal behavior from affecting world operation.
- **Lifecycle management:** Manages assets, components, and runtime resources to ensure the stability and maintainability of long-term operation.

In summary, LSJ’s technical route emphasizes that the engineering system provides authoritative execution and engineering validation, while the model system provides intelligent proposal, completion, and optimization capabilities. Models propose candidates, complete context, and organize generated results; the engineering layer handles authoritative execution, state consistency, and exception fallback.

3.3 LSJ Latent World-State Model: World-State Prediction and Candidate Structure Inference

The LSJ latent world-state model does not aim to directly generate images. Instead, it treats images, text, multimodal descriptions, and existing world context as observations of world state, and generates verifiable structural candidates through an “observation encoding - state inference - IR decoding” process: the model does not need to reconstruct all input details, but should learn abstract state representations that support prediction, understanding, and planning [4–6, 16]. The system infers latent world state and then decodes it into the structured intermediate representation (IR) required by structural world candidates, including spatial structures, object relationships, physics and navigation constraints, interaction relationships, and interaction-mechanism constraints.

In this process, the latent world-state model acts as a world-state inference module. Based on limited observations, it infers missing or implicit structures such as passable areas, occlusion relationships, interactable objects, and potential functional zones. The inference process first encodes multimodal inputs into observation representations, then predicts intermediate world-state IR and completes spatial relationships,

object relationships, and interactive affordance information. These IR results are not directly equivalent to runtime content. They are structural candidates for subsequent content generation, engineering validation, and runtime integration.

In representation, latent-state representation preserves structural information valuable to runtime, including spatial organization, object relationships, regional functions, interaction possibilities, and physical constraints. Through scene-structure graphs, interactive affordance modeling, and physical-constraint modeling, the system can propose structural candidates that satisfy current scene constraints and provide a predictable state basis for subsequent dynamic changes. Compared with ordinary generative models, the LSJ latent world-state model places greater emphasis on state consistency and local prediction capability, and provides local evolution candidates to assist engineering-system validation.

The structural candidates output by the latent world-state model still need to pass engineering-system validation, including resource availability, collision and structural support constraints, navigation connectivity, performance budgets, and behavioral validity. The model only proposes candidate states; authoritative execution and fallback strategies are determined by the engineering system. As a result, model outputs become structural world candidates for subsequent generation and integration pipelines, rather than visual candidates alone.

Compared with most world models centered on continuous visual generation or limited interaction, including recent video world simulators and generative interactive environments [7, 12, 21], the LSJ latent world-state model focuses more on whether structural candidates can enter the runtime system. New interaction mechanisms can first be expressed as objects, rules, interaction points, and interaction-mechanism constraints, then validated and executed by the engineering system. LSJ therefore cares more about “whether generated results can become part of the world and run over the long term” than merely providing continuous environmental images or limited roaming experiences.

3.4 Chronicle Memory System

An open world needs to remember not only the current state, but also the reasons for state changes, the relationships among events, and the long-term effects caused by user behavior. Work on generative agents, open-ended embodied agents such as Voyager, and recent LLM-agent surveys has shown that long-term memory, reflection, retrieval, planning, tool use, and collaboration mechanisms are important for believable behavior simulation [18, 22, 26]. LSJ introduces the chronicle memory system, a world-memory mechanism that organizes interaction behavior, regional changes, character relationships, generated content, and multi-user collaboration results into traceable world history.

Chronicle memory prevents the world from becoming a repeatedly reset scene. A construction, a conflict, a transaction, or a change in NPC relationships can all be recorded as part of world history and influence future narrative, tasks, regional states, and character attitudes. This is the key for LSJ to move from “content generation” to “world generation.”

3.5 Small Model Farm (SMF)

Small Model Farm (SMF) is a lightweight model collaborative system in LSJ for handling high-frequency, local, and stably evaluable tasks. It supports more natural, fast, and stable world operation at the user-perception level. Lightweight capabilities can provide fast feedback in candidate screening, localized decision-making, experience optimization, and risk prediction, thereby supporting various real-time operations in LSJ.

This organization allows LSJ to gain intelligent enhancement while maintaining world stability. It em-

phasizes long-term world experience rather than one-time model-output quality.

3.6 Relationships Among Three Core Model Capabilities: Latent World-State Model, Chronicle Memory System, and SMF

In LSJ’s layered model architecture, the world generation system, chronicle memory system, and SMF are three key capabilities. Here, the world generation system refers to a collection of generation capabilities centered on the latent world-state model described above and connected to structured IR and content-generation pipelines. The three capabilities solve different problems:

World generation system: responsible for recovering world-state IR from images, text, task needs, and existing world context, and proposing candidates for space, objects, physics, navigation, and interaction relationships.

Chronicle memory system: responsible for recording world operation facts, interaction behaviors, and generated results, consolidating them into long-term memory, and performing compression, completion, narrative structuring, and model internalization.

SMF: uses the results consolidated by the generation system and chronicle memory system to form high-frequency local capabilities and achieve runtime acceleration.

From the data-flow perspective, new content produced by the world generation system triggers events and state changes. The chronicle memory system consolidates these results as long-term memory and provides background constraints for subsequent generation. SMF uses outputs from the generation system and chronicle memory system to train high-frequency local models and achieve runtime acceleration.

From the control-flow perspective, outputs of the generation system must pass engineering validation before entering runtime. The chronicle memory system only organizes memory and narrative and does not replace authoritative state. SMF acts only as a validated optimization side path after rule validation and performance profiling. Together, the three form a “generate - run - record - distill - regenerate” cycle under the constraints of the engineering execution layer.

From the perspective of long-term evolution, the chronicle memory system is the core of long-term memory, the world generation system is the core of content expansion, and SMF is the core of runtime acceleration. Their collaboration enables LSJ not only to generate content, but also to convert content into history, reuse it as context, and transform context and simulation into high-frequency local capabilities, forming a continuously evolving world-intelligence framework.

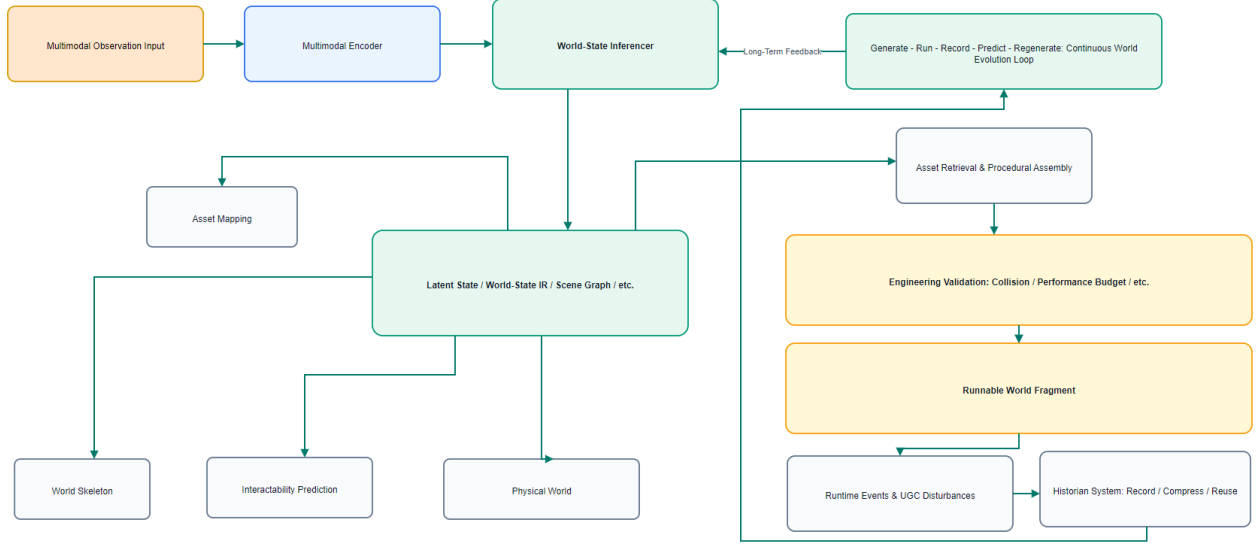


Figure 3-1: LSJ World-Model Generation and Evolution Pipeline

Figure 3-1 illustrates how multimodal observations enter latent world-state inference and form a continuously evolving closed loop through engineering validation, runtime integration, and chronicle recording.

3.7 Visual Expression Capability: Differentiated Visual Experience Under Shared Logic

LSJ’s visual scalability and extensibility are reflected not only in the scale of explorable content, but also in the diversity of world presentation. Based on the same set of resources, the system uses a real-time visual-style overlay layer to perform real-time style transformation, providing differentiated visual experiences for different participants while maintaining consistency in world logic. Object positions, physical states, task progress, NPC behavior, collision results, resource loading, event triggers, and narrative states remain synchronized, while visual style can vary according to perception preferences, character identity, regional atmosphere, narrative state, or device capability.

With the development of Neural Radiance Fields (NeRF), 3D Gaussian Splatting, and real-time neural rendering, high-performance visual models improve rendering efficiency in super-resolution, frame generation, ray-tracing denoising, temporal reconstruction, and real-time novel-view synthesis [10, 15, 19, 28]. LSJ does not rely on end-to-end models to directly generate final images. Instead, it combines model capabilities with traditional rendering pipelines to achieve controllable and stable real-time visual presentation.

This system separates the “world itself” from “visual perception.” World state is maintained by the engineering system to guarantee consistency shared by multiple users. The visual-style layer allows differentiated expression, enabling the same scene to appear in diverse styles to different participants while maintaining consistency in interaction, tasks, and physics. During long-term operation, the chronicle memory system can adjust visual expression according to events and world changes, making style evolution a visual embodiment of world history.

3.8 Overview of LSJ Standardized Model Capabilities

The models described in this section belong to LSJ’s standardized engineering models. They are engineered, trained, and deployed around tasks such as multimodal understanding, asset generation, interaction

prediction, and runtime validation. Most data and training processes are proprietary or dedicated to internal resources, mainly used to support foundational intelligence capabilities in the system.

LSJ contains multiple types of standardized engineering models, covering multimodal understanding and semantic alignment, stylized 3D asset generation, interaction and operability prediction, structured annotation of generated assets, and runtime compatibility and quality validation. These models do not replace large language models. Instead, they are embedded into the engineering path of asset generation, logic validation, and runtime integration, so generated results satisfy runtime feasibility constraints before becoming part of the world.

3.9 Semantic Addressing System

LSJ’s semantic addressing system performs intelligent routing at the unified entry point. Its core idea is to complete semantic addressing before model invocation. At the entry layer, the system semantically encodes raw input and converts natural language into high-density semantic addresses. This reduces token consumption by remote models, improves scheduling accuracy, and avoids resource waste caused by irrelevant context and repeated explanation. This role is directly related to semantic-routing methods that use semantic representations to select downstream models, tools, or network-management actions before invoking heavier reasoning components [1, 24].

Semantic addresses also handle route distribution, context compression, security-constraint injection, and capability-boundary control. Model invocation is therefore no longer a linear “direct input - direct generation” process. Instead, it becomes a multilayer collaboration of “encoded input - route distribution - strong-model generation - result validation - engineering-system takeover.” In LSJ, tokens are treated as schedulable, cacheable, and reusable computational resources for semantic processing. This enables sustainable model use in asset generation, dynamic logic, multi-user collaboration, and world evolution.

3.10 Summary

The role of layered world intelligence is to organize latent world-state inference, high-level understanding, long-term memory, lightweight localized decision-making, visual expression, semantic addresses, and engineering execution into a single set of world runtime capabilities.

4 Dynamic Multimodal and Multi-Type Content Generation with Runtime Integration for Open Worlds

4.1 Chapter Overview

LSJ’s dynamic multimodal and multi-type content generation with runtime integration capability is an open-world content-integration capability rather than a one-time content generator. Procedural content generation research in games has long studied the automatic generation of levels, rules, tasks, environments, and interactive content, while recent LLM-based work has further expanded the design space of game content generation, 3D level generation, and interaction design [11, 29]. A user’s natural language, audio, images, real-world needs, external materials, third-party API results, and world runtime events can all become inputs for content generation. These inputs become part of LSJ only after they are organized into world components that are composable, executable, and reusable.

4.2 Dynamic Multimodal and Multi-Type Content Generation with Runtime Integration Pipeline

The dynamic multimodal and multi-type content generation with runtime integration pipeline receives user needs, real-world inputs, external materials, and structural candidates proposed by the latent world-state model. It follows the main path of “structured expression - multi-type content generation - validation and runtime integration.” After intent parsing and mechanism identification, the system uses the unified intermediate representation to coordinate the generation of assets, materials, logic code, and other content.

The pipeline in this chapter is based on the layered world intelligence from the previous chapter. It receives structural candidates proposed by the latent world-state model and invokes the chronicle memory system, the semantic addressing system, and SMF to provide historical constraints, requirement routing, and candidate evaluation. Engineering execution boundaries ensure that generated content can be stably integrated into the running world.

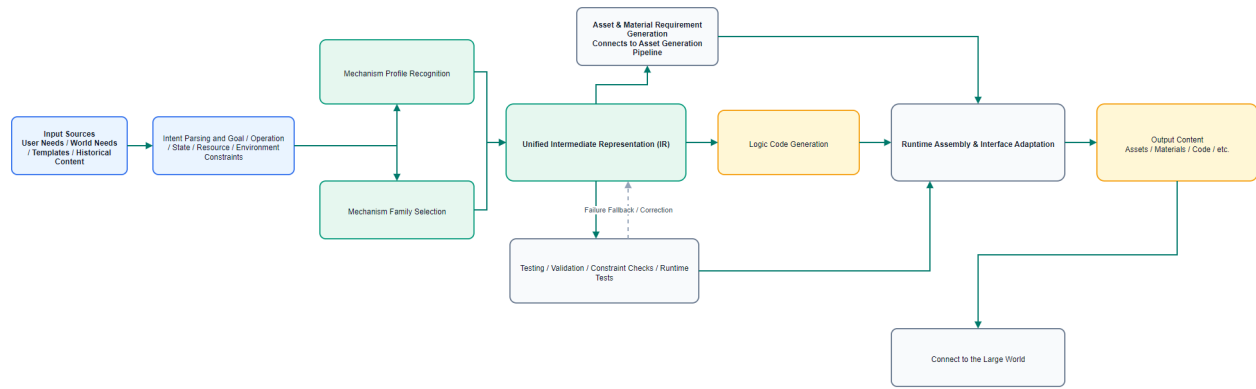


Figure 4-1: Dynamic Multimodal and Multi-Type Content Generation with Runtime Integration Pipeline for Open Worlds

Figure 4-1 illustrates how dynamic content moves from open-ended requirements into the running world. LSJ generation produces content units that can enter the main-world runtime environment through validation, assembly, and integration, rather than remaining isolated material assets.

4.3 Intent Parsing, Mechanism Identification, and Unified Intermediate Representation

Before open-ended input enters the generation layer, it needs to be organized into requirement expressions that the system can understand. Intent parsing and constraint extraction identify core goals, input-output relationships, spatial dependencies, and resource limitations. Mechanism profiles and mechanism families describe the mechanism-level characteristics and reusable categories of content requirements, allowing the system to determine which runtime structure the requirement belongs to, such as an interaction mechanism, event structure, or coordination pattern.

A mechanism profile is a structured description of interaction methods, trigger conditions, object relationships, and feedback forms in a content requirement. A mechanism family is a reusable class of mechanism templates, such as resource acquisition, structural construction, value exchange, conflict handling, collaboration, or task progression.

The unified intermediate representation (IR) is the representation layer connecting open-ended requirements, multi-source generation capabilities, and runtime integration. It organizes objects, states, rules, feed-

back, and resource requirements into the same abstraction layer, allowing textual intent, resource expression, and logical relationships to be processed uniformly.

IR is also the transition layer through which structural candidates from the latent world-state model enter the engineering world. A model can propose content intent, scene relationships, or event candidates, but these candidates must be structurally organized through IR before they can enter asset generation, logic generation, validation testing, and runtime integration.

Through a single structured abstraction layer, IR aligns natural-language intent, multimodal requirements, external API results, and real organizational information into the same semantic space, reducing fragmentation of adaptation layers and semantic inconsistency.

4.4 World-Grounded Transformation of Real-World Inputs

In LSJ, real-world input is an important source for dynamic content integration. Real-world input can come from personal needs, organizational activities, offline events, business processes, external materials, third-party system states, or multimodal information collected by users in real environments. After entering the system, such input cannot remain as ordinary attachments or explanatory text. It needs to be grounded into executable world objects.

The world-grounded transformation of real-world input contains four steps. First, the system identifies the intent, actors, time, location, organizational relationships, and constraints of the real-world input. Second, based on the unified intermediate representation (IR) described in Section 4.3, the system maps these elements into structures that the world can consume: spaces, objects, identities, permissions, tasks, rules, resource requirements, and event-trigger conditions. Third, the generation and assembly pipeline creates or updates world content based on the IR, such as event spaces, organizational spaces, task chains, NPC responsibilities, transaction rules, or collaboration processes. Fourth, the engineering system checks constraint validity, permissions, security, performance, and runtime consistency to ensure that real-world input does not damage long-term state after entering the world.

Operationally, real-world input must be interpreted in world context and transformed into structures that are interactable, recordable, and governable. This mechanism allows real-world organizations such as schools, companies, institutional actors, or collaboration groups to translate activities, members, processes, and permissions into spaces, tasks, roles, and events inside the world. The real world therefore becomes an input source for LSJ world generation and evolution, rather than a passive background.

4.5 Asset Generation Sub-Pipeline and Dual-Path Generation

The asset generation sub-pipeline receives structured requirements from the dynamic content pipeline and completes generation, assembly, post-processing, evaluation, and output around assets, materials, interaction resources, and rendering and presentation resources. Recent 3D content generation has advanced rapidly in text- or image-conditioned 3D object creation, structured 3D scenes, and explorable 3D-world generation [13, 17, 27]. In LSJ, however, the asset sub-pipeline functions as a production chain for world-content integration rather than a simple generation module.

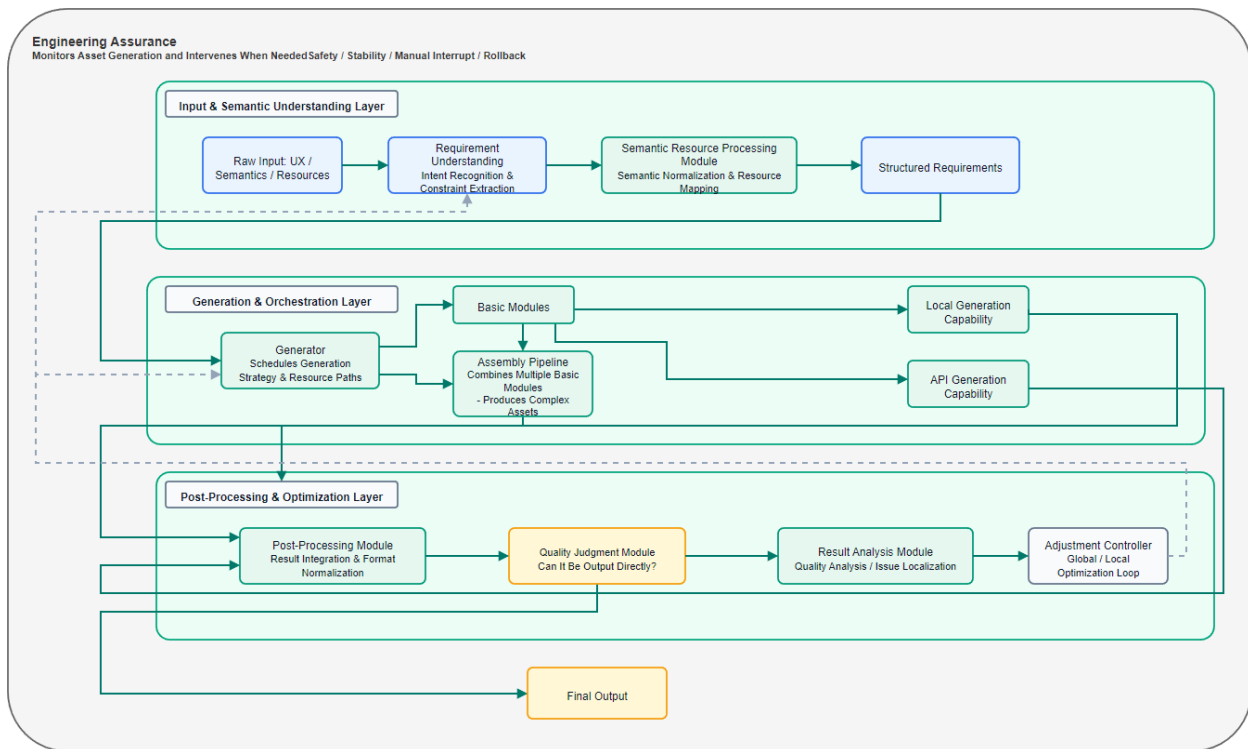


Figure 4-2: Overall Architecture of the Asset Generation System

The asset generation layer contains two types of capabilities: the base-resource generation path and the compositional asset-assembly path. Their outputs are uniformly integrated through a general optimization layer, avoiding a generation process limited to a single mode.

4.6 Basic Modules, API Capabilities, and Assembly Pipeline

Basic modules address relatively standardized content-generation requirements and can integrate both local generation capabilities and API generation capabilities. Third-party services for text, image, 3D, audio, motion, material, and code generation can all become part of content-production capability. LSJ incorporates multi-source generation capabilities into a unified world-content integration system instead of binding itself to a particular model or service.

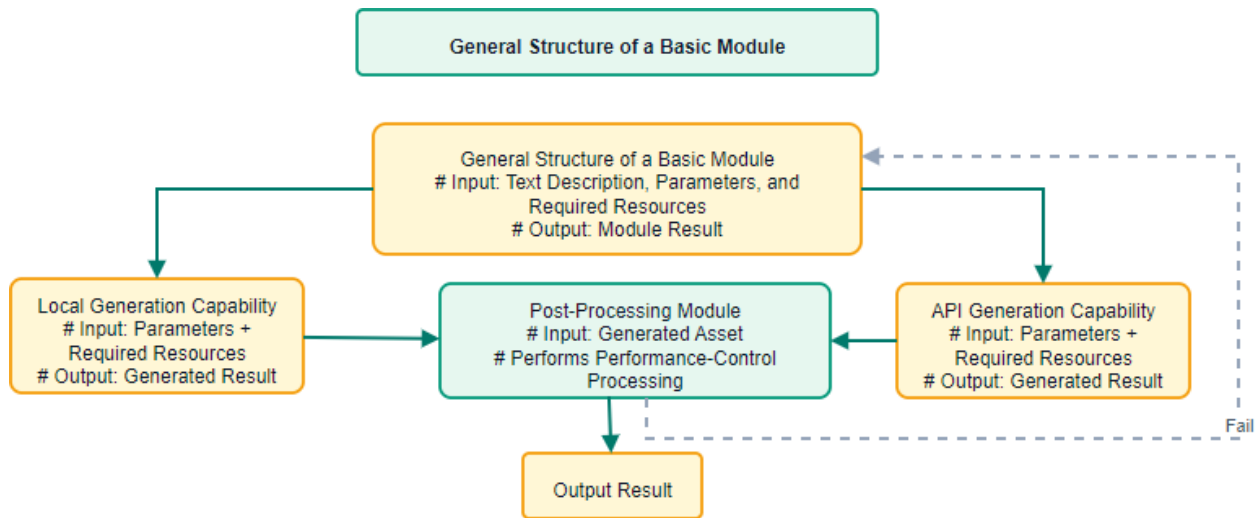


Figure 4-3: General Structure of Basic Modules

Figure 4-3 summarizes the general structure of basic modules. Local generation capabilities and API generation capabilities form parallel capability sources, and post-processing produces module results usable by upper-level pipelines. When a generation module has a localized failure or defect, the system locates the error and performs targeted repair on that module, enabling local adjustment and result improvement.

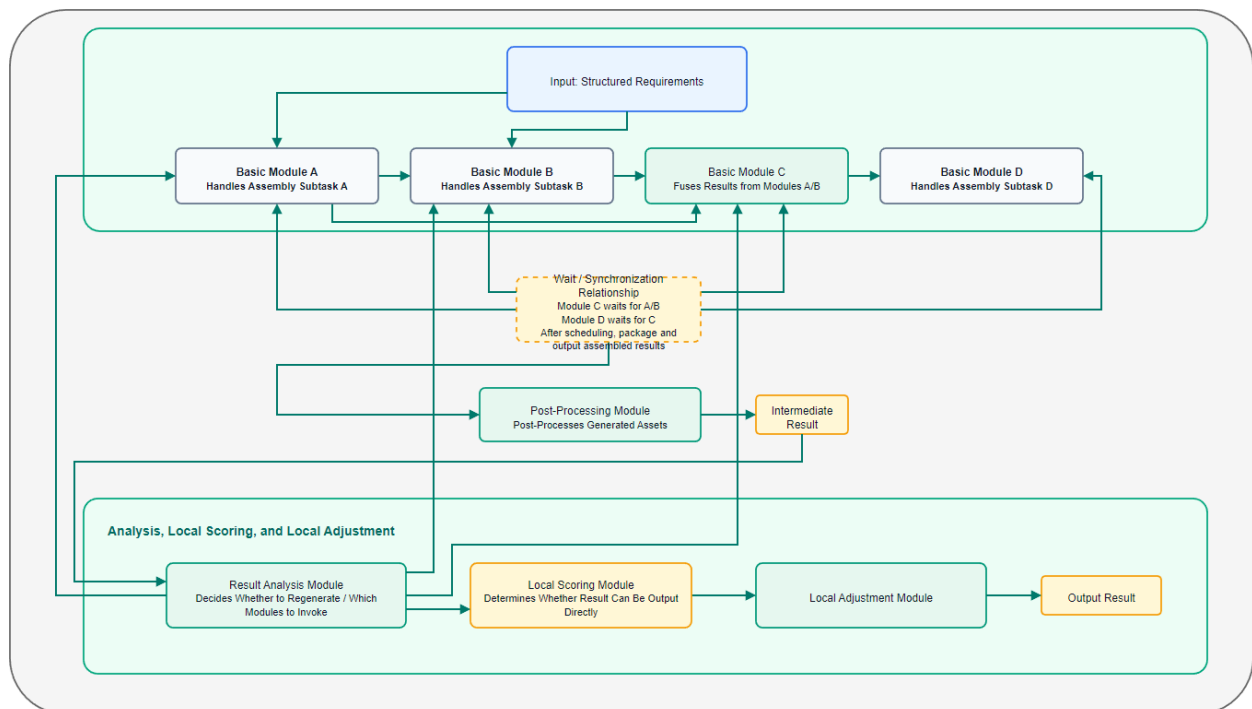


Figure 4-4: Assembly Pipeline Architecture Details

Figure 4-4 illustrates the assembly pipeline for complex asset generation, showing how multiple basic modules form more complete world-content units through composition, synchronization, caching, and fusion.

4.7 General Optimization, Feedback Loop, and Engineering Assurance

Before dynamic content enters the world, it needs unified output control. The system maintains quality thresholds and an iterative refinement loop for generated results: candidates that pass quality evaluation are directly output, while those that fail return to the generation chain for reprocessing, so content generation serves stable world operation.

Engineering assurance and external configuration synchronization provide stable support for pipeline operation. The former addresses performance, security, and human-in-the-loop intervention requirements, while the latter maintains consistency among external configuration, resource state, and runtime environment. At the capability level, dynamic content generation is a collaboration with the world runtime environment rather than an isolated model call.

4.8 Content Generation in Multi-User Collaboration

Multiple users can create, extend, repair, organize, and advance events around the same world. Unlike collaborative documents or multi-user editing tools, collaboration results in LSJ are committed to shared state and affect spaces, resources, narratives, tasks, and social relationships visible to others.

4.9 Summary

Through IR, engineering validation, and runtime integration, LSJ's generation capabilities transform from content production into world-expansion capabilities.

5 Distributed Network of Autonomous Digital Federation Nodes

5.1 Chapter Overview

From the perspective of product structure, traditional content platforms usually consist of three layers: the official platform, content creators, and ordinary users. Control over connectivity, organization, and service provision is concentrated on the official platform side. Research on networked virtual environments, partitioned virtual-world infrastructure, and distributed virtual worlds has emphasized that long-term immersive virtual spaces must address identity, interoperability, low latency, multi-user coordination, security, and governance [8, 9, 14, 25]. LSJ introduces user-autonomous nodes between the official side and content creators, extending the product structure into a four-layer relationship: "official - user-autonomous node - content creator - ordinary user."

A content creator is not a fixed role in this hierarchy. A creator can operate a UGCS, or join the autonomous space of another UGCS as an ordinary user.

From the perspective of engineering architecture, this product structure corresponds to three types of nodes: official public support nodes (LSJS), user-autonomous hosting nodes (UGCS), and user-terminal access nodes (LSJC). The responsibility boundaries of these three types of nodes are expanded in Section 5.4.

As the user-facing access point, the user-terminal access node (LSJC) connects to a specific UGCS to access the complete world experience. The virtual networking layer provides cross-node reachability, organizational boundaries, and multi-protocol communication support, making LSJ's network architecture not merely a communication topology, but an infrastructure layer for autonomous worlds and a multi-layer distributed ecosystem. Based on this design, DKI-7 proposes an architecture for a highly free and extensible virtual creation platform for global creators.

5.2 From Central Platform to User-Autonomous Hosting

Traditional content platforms are usually centered on the official platform, with both content creators and ordinary users organized around centralized platform services. Control over connectivity, organization, service provision, and distribution is mainly concentrated on the central platform. This structure is suitable for distributing content, but not for hosting large numbers of long-running, continuously evolving worlds with local rules and membership relationships; open, immersive, and sustainable virtual worlds generally require stronger network hosting, partitioning, interoperability, and social organization capabilities [8, 9, 14, 25].

LSJ introduces user-autonomous hosting nodes (UGCS) between the official platform and users. UGCS transfers the main hosting capability of autonomous spaces from a single centralized platform to the user side, rather than adding only another server role. As a result, creators, communities, organizations, and guilds no longer merely upload content; they can form long-running autonomous spaces around their own service domains.

Therefore, LSJ’s network structure can be summarized as “official necessary support + user-autonomous hosting + user-facing terminal access.” This structure delegates part of long-term world hosting, organization, and operation to user-controlled nodes.

5.3 Unified Connection Space and Local Service Domains

UGCS can become a world-hosting node because it is enabled by the unified logical connection space constructed by LSJ. This connection space allows different autonomous spaces to be discovered, accessed, and coordinated under unified network semantics while maintaining the boundaries of their respective service domains. This paper focuses on the system effect brought by this structure: the unified world is no longer hosted only by a single center, but can be jointly expanded by multiple autonomous nodes.

After an ordinary user connects to a UGCS, the user does not enter an independent world separated from LSJ. Instead, the user enters the autonomous space provided by that node under LSJ’s unified world framework. The official layer of the LSJ world continues to progress under a unified worldview, macro history, and social structure, and through content synchronization and distribution mechanisms under global coordination, official foundational content, macro events, and public states remain consistent across different autonomous spaces. Users can both access unified world content provided by LSJ and obtain personalized content, services, and community interaction experiences within autonomous spaces oriented around that service domain.

Under this structure, two types of relationships exist simultaneously in LSJ. One is the public coordination relation for overall world coordination; the other is the domain-specific service relation for specific autonomous spaces. The former ensures that users, nodes, and public capabilities jointly exist in the LSJ world network. The latter ensures that each autonomous space can form a relatively independent service domain around its own members, content, NPCs, transactions, and rules.

This design allows LSJ to gain two capabilities at the same time: local autonomy and global coordination. Local autonomy allows each digital federation node to develop its own space, history, rules, and social relationships. Global coordination allows discovery, migration, transactions, collaboration, and real-world business linkage to occur among different autonomous nodes.

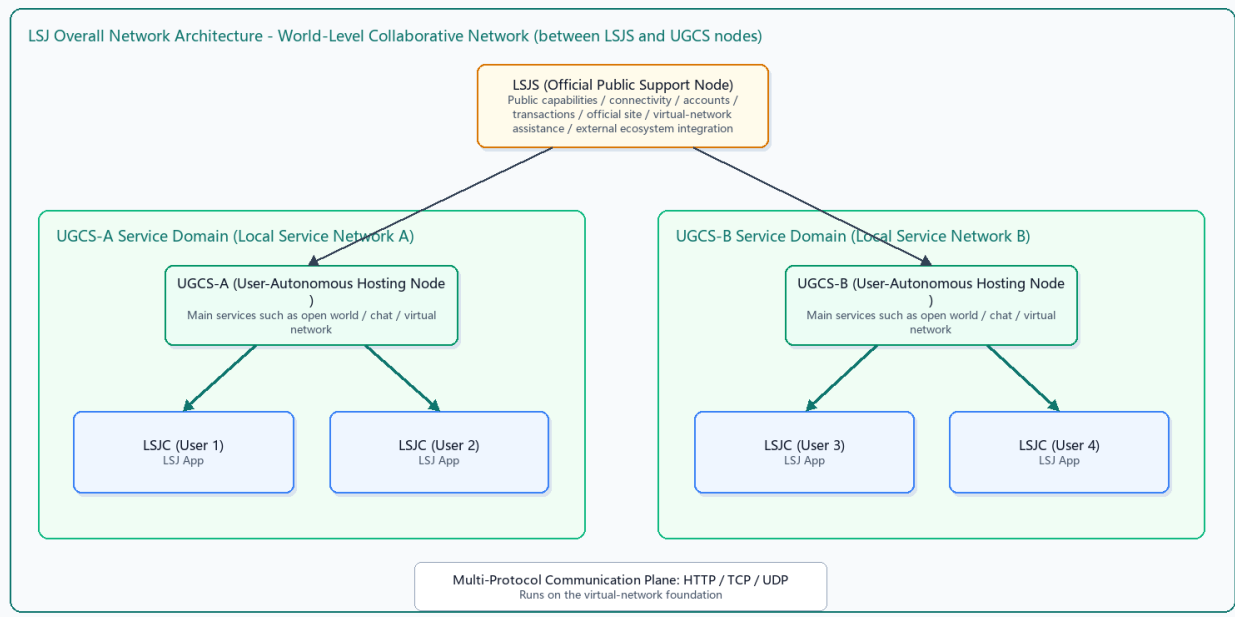


Figure 5-1: Overall Architecture of the LSJ World Network

Figure 5-1 illustrates the interconnections among LSJS, UGCS, and LSJC in the unified virtual network. LSJ's network structure is a multi-layer collaborative network composed of official public support, user-autonomous hosting, and terminal access.

5.4 Responsibility Boundaries of the Three Node Types

LSJS, UGCS, and LSJC constitute the three core node types in the LSJ network architecture. They are world-network roles with different responsibilities, not servers of different sizes.

LSJS is located at the public support layer. It undertakes responsibilities such as account management, transaction services, discovery, public coordination, external ecosystem integration, and overall connectivity assurance. It provides stable public capabilities for the entire LSJ ecosystem so user-autonomous spaces can operate under a unified order.

UGCS is located at the autonomous service-hosting layer. It organizes autonomous spaces around specific communities, organizations, or service domains and can host spaces, content, NPCs, communication services, local rules, community relationships, and member activities. UGCS is the key node through which LSJ implements user-autonomous capability in the network structure, and it is also the foundation on which an “autonomous digital federation node” can exist.

LSJC is located at the terminal experience layer. It is the entry through which users enter the world, organizing the large-scale 3D world, communication and transaction services, content flow, local intelligence capability, and external services into a composite experience perceptible to users. What users see in LSJC is the world experience, but behind it that experience is jointly formed by LSJS public support and UGCS local hosting.

Although LSJS, UGCS, and LSJC all connect to the LSJ network, their internal services, business components, shared components, and external ecosystem integration capabilities have clear responsibility boundaries. LSJS leans toward public support and global coordination, UGCS leans toward autonomous-space hosting and community operation, and LSJC leans toward terminal-experience orchestration and user

interaction.

5.5 Operational Form of Autonomous Digital Federation Nodes

From the perspective of operational form, a UGCS can internally contain spaces, members, rules, NPCs, transactions, memory, and governance methods. Different autonomous nodes can have different themes, cultures, organizational forms, and operating goals.

Creators can operate their own autonomous spaces, communities can have their own public spaces, guilds can form long-term strongholds, and schools and enterprises can build virtual organizations linked with real-world activities and organizational processes. They maintain local autonomy while entering the larger world network through LSJ public capabilities.

From the perspective of real-world integration, UGCS can be understood as a runtime host for real-world organizations within the LSJ network. Membership relationships, organizational permissions, activity schedules, business processes, and governance rules in reality can be mapped into identity systems, spatial partitions, task permissions, transaction rules, and event processes inside a UGCS. They can also connect with other organizations through the account, discovery, transaction, and public-coordination capabilities provided by LSJS.

5.6 Balance Between Official Support and User Autonomy

LSJ is not equivalent to a fully decentralized network, nor is it equivalent to a centralized content platform. Through LSJ, DKI-7 keeps accounts, discovery, transactions, and public coordination in the official support layer, while delegating part of service hosting, organization, and operation capabilities to UGCS. This maintains public order while allowing organizations to share the cost of long-term world operation.

This balance is especially important for long-term worlds. Foundational content, accounts, transactions, discovery, public coordination, and external ecosystems require stable support. Spaces, members, NPCs, content, and local relationships need to allow long-term operation by users and communities. Only when both conditions hold can the world have both order and capacity for long-term evolution.

Thus, the core value of LSJ's network architecture lies in changing how the world is organized. It shifts the world from being hosted by a single platform to being jointly hosted by multiple autonomous service domains, and it allows users to go beyond being content consumers and uploaders to become operators, organizers, and co-builders of autonomous spaces.

5.7 Summary

LSJ forms a distributed network of autonomous digital federation nodes composed of official public support nodes, user-autonomous hosting nodes, and user-terminal access nodes. Under a unified world framework, this network connects multiple sustainably operated autonomous spaces and provides a hosting foundation for multi-user collaboration, creator spaces, real-organization integration, and long-term world history.

6 Continuously Streamed Open World and Reliable Runtime Framework

6.1 Chapter Overview

The LSJ world is not a set of scenes within fixed boundaries, but an open world that can expand continuously, stream and load content hierarchically, and run for the long term while preserving traces left by

users, NPCs, tasks, and multi-user collaboration.

This runtime framework depends on five core capabilities: multi-level world representation, continuously expanding spatial organization, physically grounded feedback, state consistency, and world-state write-back.

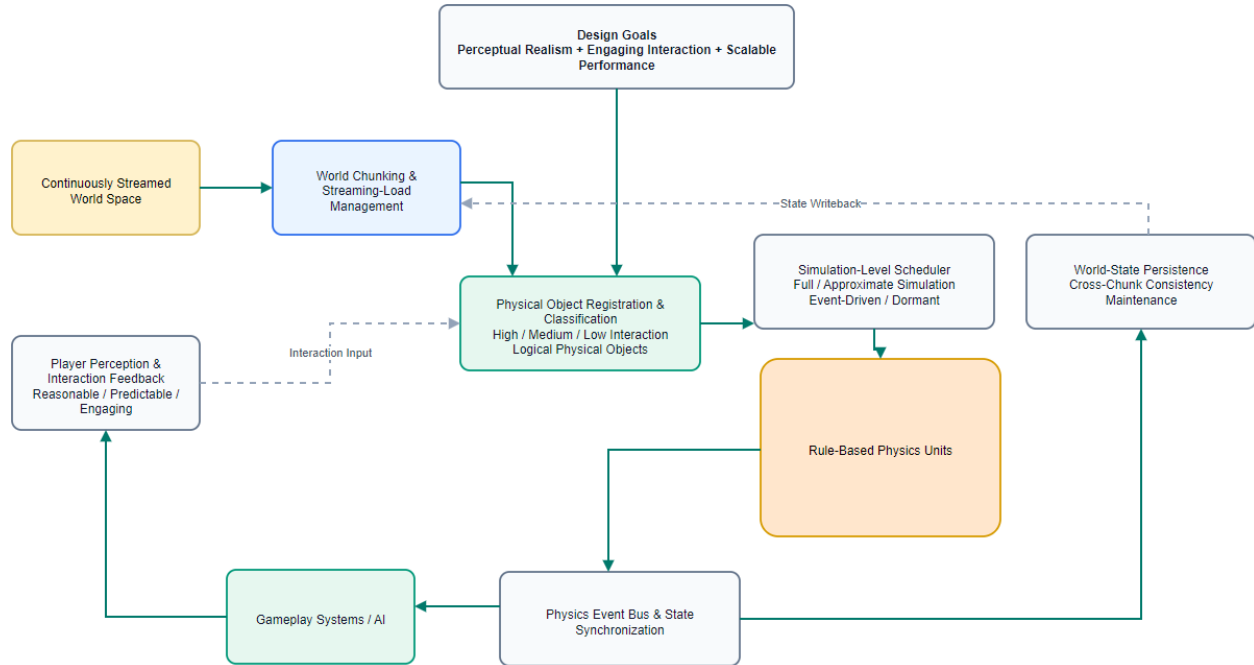


Figure 6-1: Overall Relationship of the Continuously Streamed Open World and Reliable Runtime Framework

Figure 6-1 relates spatial organization, physical feedback, runtime scheduling, and world-state writeback in LSJ’s open-world runtime.

6.2 Multi-Level World and Continuous Expansion

LSJ organizes surface, underground, outer-space, and other spatial layers under a unified world model. “Infinite” does not mean that all content must exist locally at once; the world can remain continuous and explorable through continuous expansion and on-demand operation.

6.3 Physically Grounded and Perceptual Feedback

In an open world, physical feedback must be perceptible rather than merely simulated in the background. Movement, structural change, environmental effects, regional events, and interaction behavior should produce outcomes that users can recognize and that the system can record as world state when necessary. Physically grounded feedback is therefore broader than collision or gravity: changes caused by actions should remain visible, understandable, and consequential in later experiences.

6.4 Real-World Constraints and Spatial Semantic Alignment

When real-world input is brought into LSJ, the continuously streamed open world and reliable runtime framework must align physical-world constraints with virtual space. Digital-twin and virtual-real interpromo-

tion research emphasizes that virtual systems need persistent connections among real entities, virtual models, data transmission, and semantic mapping. Industrial digital-twin platforms and recent investigations delve into 3D collaborative environments and large language models for continuously updating digital twins in changing environments [2, 20, 23, 30]. Real-world locations, scales, spatial relationships, organizational boundaries, and activity rules cannot remain as textual descriptions; they must be transformed into spatial semantics and runtime constraints in the world. For example, a classroom, exhibition booth, meeting room, neighborhood, or project space in reality needs to become navigable areas, interactable objects, permission boundaries, task entrances, and event-trigger regions in LSJ.

This alignment does not require LSJ to copy the real world in full. Real-world input must remain understandable, verifiable, and operational inside the virtual world. Scene generation, physical proxies, navigation areas, and interaction points need to be linked consistently so that translated real-world activities still make spatial sense. If those constraints are detached from virtual space, real-world integration becomes little more than static display. Real-world activities become executable world components only when location, scale, permissions, events, and feedback can operate as a closed loop inside the world.

6.5 Rule-Based Physics Capability

For an open world to keep expanding, physics cannot remain a collection of isolated effects; it must become a set of reusable world-level capabilities callable by the content system. LSJ organizes basic physics, structural change, collective behavior feedback, and environment-related feedback into higher-level rule-based capabilities that can be shared by generated content, task events, and world operation.

The reliable runtime framework can then support manually designed content, dynamic content generation, and multi-user collaboration through the same physical capability layer. Generated spaces, mechanisms, objects, and events can produce plausible feedback in the world instead of remaining at the static-display layer.

6.6 State Consistency and World Traces

A central challenge for an open world is keeping changes valid after they occur. LSJ therefore emphasizes world-state consistency: when necessary, interaction behavior, physical feedback, task progress, NPC behavior, and multi-user collaboration results must be written back into later world state.

LSJ therefore differs from a conventional large map or sandbox scene: state traces provide consistent context for subsequent events, regional changes, and multi-user experiences.

6.7 Summary

In LSJ, open-world scale is tied to persistence: spatial expansion, physical feedback, and state writeback allow the world to grow without losing the traces of long-term change.

7 World Structure, Runtime Mechanisms, and Continuous Narrative Evolution

7.1 Chapter Overview

LSJ views world evolution as a long-term process shaped by structure, runtime mechanisms, narrative trajectories, memory, and model reasoning. Recent work on generative agents, open-ended embodied agents, and LLM-based multi-agent systems has explored how memory, reflection, planning, social interaction, tool

use, and multi-agent coordination can support more believable behavior simulation [3, 18, 22, 26]. User behavior in instant messaging (IM), the large-scale 3D world, or task systems not only triggers NPC conversations and local events, but also affects character relationships, organizational situations, regional states, and world history.

For that consistency to hold over time, the world engine, rule engine, and memory-graph engine must work with the layered intelligence architecture introduced in Chapter 3. Together they organize NPC behavior, social structure, historical events, and engineering execution into the same evolution feedback loop.

The key question is what happens after a behavior candidate is proposed: how it is checked, executed, recorded, and reused as context for later world evolution.

7.2 Three Engines and Three Types of Artifacts

The world engine produces world narrative packages and social-structure packages. These artifacts describe event progression, relationship changes, and world state updates.

The rule engine turns social rules and interaction-mechanism constraints into rules that can be executed by the system. Its role is to keep behavior within valid social, mechanical, and runtime boundaries.

The memory-graph engine keeps long-term world state available for later use. It recalls the context needed for NPC decisions and regional changes, so behavior is not generated from an isolated prompt.

Together, these engines keep NPC behavior tied to social structure, history, and runtime rules. An NPC is therefore treated less as a one-off dialogue agent and more as a persistent actor inside the world.

7.3 Long-Term Evolution of World Structure, Runtime Mechanisms, and Narrative

Artifacts generated by world creation are folded back into the structured world state. Narrative packages record event trajectories and narrative progression. Social-structure packages capture organizational and character relationships. Rule packages define executable constraints, while chronicle memory records validated canonical events. At runtime, the semantic addressing mechanism recalls the context needed to keep NPC responses consistent and controllable.

NPC behavior may begin with interaction input, world events, or system events. The entry layer first generates semantic addresses. The entry layer generates semantic addresses, which the long-term semantic layer or large language model uses to propose actions, updates, triggers, or memory writes. The engineering execution layer validates these proposals; invalid candidates are rejected, simplified, or regenerated. Valid results are recorded by the chronicle memory system and can be used for lightweight model training.

Continuous NPC evolution is part of the broader long-term state-management problem, constrained by social structure, historical causality, runtime adjustment, and engineering execution. It remains constrained by social structure, historical causality, runtime adjustment, and engineering execution.

7.4 Multi-User Collaboration and Social Impact

Construction, conflict, transactions, and task progression through multi-user collaboration enter world history, influencing later character behavior, regional state, and event progression.

7.5 Summary

NPCs and world events should be managed as part of long-term world state. NPCs and world events must be managed as part of long-term world state, recordable, verifiable, and persistent, not one-time conversations or localized narrative episodes.

8 Conclusion

World creation is not merely generating more content. It is allowing space, rules, characters, memory, and real-world activities to grow over the long term within the same system.

LSJ advances a vision in which, for the first time, a persistent virtual world is treated not merely as generated content, but as a continuously evolving world system that can run in parallel with the real world, be entered and changed by users, preserve the traces of those changes, grow continuously, and remain connected with real-world activities.

9 LLM Usage Statement

The author(s) used large language models for language polishing and clarity improvement. The author(s) reviewed and take full responsibility for all content, claims, references, and conclusions.

References

- [1] Khaled Alkiek, Adlen Ksentini, Abbas Bradai, and Abd-Elhamid M. Taha. Semantic routing for enhanced performance of llm-assisted intent-based 5g core network management and orchestration, 2024. URL <https://arxiv.org/abs/2404.15869>.
- [2] Harry Amad, Nicolás Astorga, and Mihaela van der Schaar. Continuously updating digital twins using large language models, 2025. URL <https://arxiv.org/abs/2506.12091>.
- [3] R. M. Aratchige and W. M. K. S. Ilmini. Llms working in harmony: A survey on the technological aspects of building effective llm-based multi agent systems, 2025. URL <https://arxiv.org/abs/2504.01963>.
- [4] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023. URL <https://arxiv.org/abs/2301.08243>.
- [5] Mahmoud Assran et al. V-jepa 2: Self-supervised video models enable understanding, prediction and planning, 2025. URL <https://arxiv.org/abs/2506.09985>.
- [6] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video, 2024. URL <https://arxiv.org/abs/2404.08471>.
- [7] Jake Bruce, Michael D. Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, Yusuf Aytar, Sarah Bechtle, Feryal Behbahani, Stephanie C. Y. Chan, Nicolas Heess, Luis Gonzalez, Simon Osindero, Sherjil Ozair, Scott Reed, Jingwei Zhang, Konrad Zolna, Jeff Clune, Tim Rocktäschel, Andrew Welchman, and Andrei A. Rusu. Genie: Generative interactive environments, 2024. URL <https://arxiv.org/abs/2402.15391>.
- [8] Juan Cabrera, Juan Ruiz, Manuel Fernandez, Pablo Martinez, and Antonio F. Skarmeta. Key technologies for networked virtual environments, 2021. URL <https://arxiv.org/abs/2102.09847>.

- [9] Ben Y. Chang, Jae Hwang, Ravi Jain, Robert Morris, Joshua Reich, and David Tang. Pvw: Designing virtual world server infrastructure. In *Proceedings of the 9th ACM SIGCOMM Workshop on Network and System Support for Games*, 2010. doi: 10.1109/NETGAMES.2010.5680988. URL https://pdxscholar.library.pdx.edu/compsci_fac/217/.
- [10] Guikun Chen and Wenguan Wang. A survey on 3d gaussian splatting, 2024. URL <https://arxiv.org/abs/2401.03890>.
- [11] Mahdi Farrokhi Maleki and Richard Zhao. Procedural content generation in games: A survey with insights on emerging llm integration. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, volume 20, pages 167–178, 2024. doi: 10.1609/aiide.v20i1.31877. URL <https://doi.org/10.1609/aiide.v20i1.31877>.
- [12] Google DeepMind. Genie 2: A large-scale foundation world model. Google DeepMind Research Blog, 2024. URL <https://deepmind.google/discover/blog/genie-2-a-large-scale-foundation-world-model/>.
- [13] HunyuanWorld Team, Zhenwei Wang, Yuhao Liu, Junta Wu, Zixiao Gu, Haoyuan Wang, Xuhui Zuo, Tianyu Huang, Wenhuan Li, Sheng Zhang, Yihang Lian, Yulin Tsai, Lifu Wang, Sicong Liu, Puhua Jiang, Xianghui Yang, Dongyuan Guo, Yixuan Tang, Xinyue Mao, Jiaao Yu, Junlin Yu, Jihong Zhang, Meng Chen, Liang Dong, Yiwen Jia, Chao Zhang, Yonghao Tan, Hao Zhang, Zheng Ye, Peng He, Runzhou Wu, Minghui Chen, Zhan Li, Wangchen Qin, Lei Wang, Yifu Sun, Lin Niu, Xiang Yuan, Xiaofeng Yang, Yingping He, Jie Xiao, Yangyu Tao, Jianchen Zhu, Jinbao Xue, Kai Liu, Chongqing Zhao, Xinming Wu, Tian Liu, Peng Chen, Di Wang, Yuhong Liu, Linus, Jie Jiang, Tengfei Wang, and Chunchao Guo. Hunyuanworld 1.0: Generating immersive, explorable, and interactive 3d worlds from words or pixels, 2025. URL <https://arxiv.org/abs/2507.21809>.
- [14] Improbable. Spatialos: World-building technology for persistent online worlds. Industrial technical announcement, 2015. URL <https://www.improbable.io/>.
- [15] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 2023. doi: 10.1145/3592433. URL <https://arxiv.org/abs/2308.04079>.
- [16] Yann LeCun. A path towards autonomous machine intelligence. OpenReview, 2022. URL <https://openreview.net/pdf?id=BZ5a1r-kVsf>.
- [17] Yuhan Li et al. A comprehensive survey on 3d content generation, 2024. URL <https://arxiv.org/abs/2402.01166>.
- [18] Junyu Luo, Weizhi Zhang, Ye Yuan, Yusheng Zhao, Junwei Yang, Yiyang Gu, Bohan Wu, Binqi Chen, Ziyue Qiao, Qingqing Long, Rongcheng Tu, Xiao Luo, Wei Ju, Yifan Wang, Xiao Meng, Chenwu Liu, Jingyang Yuan, Shichang Zhang, Yiqiao Jin, Fan Zhang, Xian Wu, Hanqing Zhao, Dacheng Tao, Philip S. Yu, and Ming Zhang. Large language model agent: A survey on methodology, applications and challenges, 2025. URL <https://arxiv.org/abs/2503.21460>.
- [19] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. In *European Conference on Computer Vision*, pages 405–421, 2020. doi: 10.1007/978-3-030-58452-8_24. URL <https://arxiv.org/abs/2003.08934>.

- [20] NVIDIA. Nvidia omniverse documentation. NVIDIA Developer Documentation, 2024. URL <https://docs.nvidia.com/omniverse/>.
- [21] OpenAI. Video generation models as world simulators. OpenAI Research, 2024. URL <https://openai.com/index/video-generation-models-as-world-simulators/>.
- [22] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023. URL <https://arxiv.org/abs/2304.03442>.
- [23] Christian Plóciennik et al. Virtual commissioning of linked cells using digital models in an industrial metaverse. *Automation*, 5(1):1–17, 2024. doi: 10.3390/automation5010001. URL <https://www.mdpi.com/2673-4052/5/1/1>.
- [24] Lingfeng Shen, Shujian Huang, Xuechen Luo, Zhen Chen, Yichao Liu, Ming Li, Xiang Zhang, Hong Sun, and Yuxuan Wang. vllm semantic router: Signal driven decision routing for mixture-of-modality models, 2026. URL <https://arxiv.org/abs/2603.04444>.
- [25] Sandeep Singhal and Michael Zyda. *Networked Virtual Environments: Design and Implementation*. Addison-Wesley, 1999. ISBN 0201325578. URL https://books.google.com/books/about/Networked_Virtual_Environments.html?id=Oq5QAAAAMAAJ.
- [26] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023. URL <https://arxiv.org/abs/2305.16291>.
- [27] Ling Wang, Hao-Xiang Guo, Xinzhou Wang, Fuchun Sun, Kai Sun, Pengkun Liu, Hang Xiao, Zhong Wang, Guangyuan Fu, Eric Li, Yang Liu, and Yikai Wang. Scenetransporter: Optimal transport-guided compositional latent diffusion for single-image structured 3d scene generation, 2026. URL <https://arxiv.org/abs/2602.22785>.
- [28] Yuting Xiao, Zhong Xue, Tianyu Tang, Yiqi Ge, Zhen Miao, Huan Zhang, Zhongang Cai, Kui Yang, Fan Yang, Zhiguo Cao, Hongbin Zha, and Lei Zhang. Neural radiance fields for the real world: A survey, 2025. URL <https://arxiv.org/abs/2501.13104>.
- [29] Kaijie Xu and Clark Verbrugge. A database-driven framework for 3d level generation with llms, 2025. URL <https://arxiv.org/abs/2508.18533>.
- [30] Qianqian Zhang et al. Semantic-aware digital twin for metaverse: A comprehensive review, 2023. URL <https://arxiv.org/abs/2305.18304>.